Methodology

This combined methodology document collects the experimentation setup, architecture, RAG pipeline, analytics procedures, and evaluation protocol used for FE-AI (BGSU — ECE & ROBO). It is intended for inclusion in a Methods section of a research paper and as a reproducibility checklist.

1. Experimentation Setup

1.1 Research Questions

- How accurately can the system detect student misconceptions from short free-text student submissions?
- What retrieval strategy (embedding + FAISS vs. TF-IDF) provides the most relevant context for downstream generation tasks in an offline-first setting?
- How do local LLMs (Ollama) perform on Bloom level classification and misconception labeling when operated fully locally?

1.2 Datasets and Splits

- Source logs: `logs/activity_log.json` — all recorded student interactions and assistant responses.
- RAG corpus: `data/vector_index/<COURSE>/docs.jsonl` (ECE, ROBO).
- Analytics outputs: `data/analytics/*` produced by `scripts/*`.
- For experiments, sample a stratified dataset per course by topic and difficulty:
  - Train/dev/test split: 70/15/15 by student-session.
  - Hold-out evaluation: ensure no overlap of students across splits to avoid leakage in per-student metrics.

1.3 Baselines and Variants

- Retrieval:
  - Embedding-based: local embeddings + FAISS (default pipeline).
  - TF-IDF baseline: scikit-learn TF-IDF with cosine similarity (fallback pipeline).
- Labeling & Classification:
  - Local Ollama models (e.g., `llama3.2:3b`, `qwen3:4b`) via HTTP (OLLAMA_URL). All model calls are performed locally; the system is designed to avoid outbound HTTPS calls to external LLM providers.
- Ablations:
  - Top-1 vs Top-K retrieval.
  - Similarity threshold vs no-threshold (e.g., drop docs < 0.90 cosine similarity when using embeddings).

1.4 Metrics

- Misconception detection: Precision, Recall, F1 (binary + per-subtopic macro-F1).
- Subtopic classification: micro-F1 / macro-F1.
- Retrieval: Recall@1, Recall@k, MRR@k.
- Dashboard and aggregate measures: topic difficulty ranking correlation with human judgments (Spearman rho), instructor-rated usefulness (Likert).

1.5 Experimental Protocols

- For each experimental run:
  1. Rebuild or load RAG indexes (see `scripts/build_rag.py`).
  2. Run ingestion scripts to produce analytics artifacts: `python scripts/compute_counts.py`, `python scripts/compute_topic_difficulty.py`, `python scripts/compute_errors.py`.
  3. Sample evaluation examples from `logs/activity_log.json` using consistent random seeds.
  4. Run classifiers with fixed seeds where applicable; repeat experiments with 3 different seeds and report mean ± std.

2. System Architecture (brief)

- Frontend: Streamlit pages in `pages/*` and `lib/ui.py`.
- Generation abstraction: `lib/chatgeneration.py` — wraps provider selection, rate limiting, and respects the runtime HTTP guard in `lib/disable_http.py`.
- Retrieval: `lib/rag.py` — embedding + FAISS primary path; TF-IDF via scikit-learn fallback.
- Analytics: `lib/analytics_helpers.py`, `lib/analytics_agent.py`, `scripts/*` perform offline batch processing.
- Design notes: the system's retrieval-augmented generation (RAG) pipeline is configured to supply curriculum-aligned, contextful information for classroom content. Retrieved passages include pedagogical metadata (topic, subtopic, lecture id) so downstream components — classifiers, visualizations, and the instructor dashboard — can be customized by instructors (for example by restricting retrieval to specific modules or weighting recent lecture materials).
- Analytics architecture: the analytics dashboard composes small, focused agents (topic classifier, misconception detector, Bloom scorer, aggregator, visualizer). These agents exchange intermediate results via JSON artifacts stored in `data/analytics/*`, enabling modular reprocessing, traceability, and controlled ensemble experiments.

3. Retrieval & Knowledge Base

3.1 KB Layout

- Files: `data/vector_index/<COURSE>/docs.jsonl` and `index.faiss` per course.
- Document format: newline delimited JSON with fields `id`, `text`, `meta` (topic, subtopic, source).

3.2 Embeddings + FAISS

- Default embedding model: `all-minilm:latest` (local embedding model) used for local embedding computations and configured in `lib/rag.py`.
- FAISS index: built via `scripts/build_rag.py` and stored under `data/vector_index/<COURSE>/index.faiss`.
- Retrieval flow:
  - Query -> embed -> FAISS search -> top-K candidates -> similarity scoring -> optional thresholding.
- Configured behavior: top-1 retrieval for coding companion, and drop documents with similarity < 0.90 when configured.
- Details and operational notes:
  - Preprocessing: documents are cleaned (whitespace, simple HTML removal), chunked into 200–500 token passages (configurable), and normalized before embedding.
  - Embedding batching: embeddings are computed in batches to reduce memory overhead; batch size is set in `lib/rag.py` and should be tuned for your host CPU/RAM.
  - Embedding dimension & index: `all-minilm:latest` produces fixed-length vectors (ensure FAISS index type matches the vector dimension). We recommend using an IVF+PQ or HNSW index for larger corpora; `scripts/build_rag.py` contains the index construction parameters.
  - Index maintenance: rebuild the FAISS index after significant corpus updates; optionally schedule incremental rebuilds for frequent updates.
  - Storage & paths: indexes and docs are stored under `data/vector_index/<COURSE>/` — archive these artifacts alongside `requirements.txt` for reproducibility.
  - Model provenance: pin the `all-minilm:latest` artifact or record its local path/digest used in experiments to ensure reproducibility.
  - Local-only operation: all embedding computations are performed locally using the `all-minilm:latest` model; the system does not make outbound HTTPS calls to remote embedding or model services. Ollama is used as a local model server (default via `http://localhost:11434`) and the runtime HTTP guard blocks external HTTP by default.
  - RAG usage for instructional context: the FAISS-backed retrieval is

used not merely as a generic vector database but as a curriculum-aware context layer. Each indexed passage carries instructional metadata and is surfaced to generation and classification routines to improve relevance and interpretability for classroom use. This design decision was made to enable instructor-driven customization (e.g., limiting retrieval to a syllabus module, prioritizing recent lecture notes, or surfacing canonical examples), and to make dashboard outputs actionable for course staff.

3.3 TF-IDF Fallback

- Implemented in `lib/rag.py` using scikit-learn's `TfidfVectorizer` and cosine similarity.
- Used when embeddings or local model resources are unavailable (offline mode).

4. Models, Prompts, and Labeling

4.1 Misconception Detection

- Approach: present student submission and (optionally) system answer/context to a classifier prompt asking whether a misconception is present and to assign a subtopic.
- Implementation: `scripts/compute_errors.py` orchestrates generation calls through `lib.chatgeneration.generate_chat_completion()`.
- Label schema: `{misconception: true|false, subtopic: <string>|null, confidence: <0..1>}`.

4.2 Bloom Level Classification

- Implementation lives in `lib/analytics_agent.py` as `classify_bloom()`.
- Prompted JSON schema: the model returns a Bloom taxonomy label (Remember, Understand, Apply, Analyze, Evaluate, Create) with an optional score.

4.3 Prompts & Tempering

- All prompts are stored or constructed in code near their caller functions (see `lib/analytics_agent.py` and `scripts/classify_topics_ollama.py`).
- Control temperature and tokens via config constants in `config.py`.

5. Evaluation Protocols (detailed)

5.1 Misconception Detection Evaluation

- Ground truth: human annotation of a randomly sampled subset (N >= 500 examples across courses and topics).

- Annotation process: two annotators label independently; adjudicate disagreements by a third annotator.
- Metrics reported: precision, recall, F1 for misconception detection. Also per-subtopic confusion matrices.

5.2 Subtopic Classification Evaluation

- Use micro and macro F1 against human-subtopic labels.
- Evaluate top-k candidate subtopic suggestions (e.g., top-3) for recall.

5.3 Retrieval Evaluation

- Build a test set with queries and known relevant documents.
- Compute Recall@1, Recall@5, MRR@5 for embedding-based vs TF-IDF retrieval.
- If human relevance labels are costly, use surrogate measures like overlap with ground-truth doc IDs that generated the assistant content.

5.4 Statistical Testing

- Use paired tests across variants (e.g., Wilcoxon signed-rank for paired F1 scores) and report effect sizes (Cohen's d where applicable).

6. Experimental Procedures (step-by-step)

6.1 Reproducible Run (single experiment)

- Activate environment (example if using venv):

```
source .venv/bin/activate
pip install -r requirements.txt
```

- Rebuild RAG index if necessary:

```
python scripts/build_rag.py --course ECE
python scripts/build_rag.py --course ROBO
```

- Generate analytics artifacts used in experiments:

```
python scripts/compute_counts.py
python scripts/compute_topic_difficulty.py
python scripts/compute_errors.py
```

- Run evaluation harness (example test script you should create to run your labeled test suite):

```
python experiments/run_evaluation.py --config experiments/configs/misconception_eval.yaml
```

(If `experiments/` doesn't exist yet, create a small runner that loads test examples and calls `lib.chatgeneration` + `lib/rag`.)

6.2 Model Comparison / Ablation

- For each model variant:
  - Set `OLLAMA_URL` in the environment to point to the local Ollama daemon (default `http://localhost:11434`). This project avoids outbound HTTPS calls to external LLM providers; all model calls should be local.
  - Run the same evaluation harness with identical test sets and seeds.
  - Collect logs to `logs/` and analytics outputs to `data/analytics/`.

6.3 Local Wi-Fi Server Setup (access from other devices)

- Purpose: run the app and make it reachable from other devices on the same local Wi-Fi network.
- Steps:
  1. Find the host machine's local IP address (one of these will work):

```
hostname -I
ip addr show
```

  2. Open the Streamlit server to the local network by binding to `0.0.0.0` (or the specific local IP) and choosing a port (default 8501):

```
source .venv/bin/activate
streamlit run chatgpt.py --server.address 0.0.0.0 --server.port 8501
```

  3. Allow the port through the host firewall (example for `ufw`):

```
sudo ufw allow 8501/tcp
sudo ufw reload
```

  4. Ensure the client device is on the same Wi-Fi network and open a browser to:

```
http://<HOST_LOCAL_IP>:8501
```

  5. If you need access beyond the LAN, set up router port forwarding for the chosen port and secure the host (this is NOT recommended without additional security measures).

Notes: - Keep `OLLAMA_URL` pointing to the local daemon (e.g., `http://localhost:11434`) and run Ollama on the host. The runtime HTTP guard in the app prevents outbound HTTP by default except for configured local allowlist entries. - For temporary testing, disabling strict firewall rules is fine on trusted networks, but for longer-term usage secure the host and consider VPN or SSH tunneling.

7. Hardware and Environment

- Record hardware specs with `uname -a`, `lscpu`, `free -h` and include them as an experimental appendix.

- Example machine used in development: Intel Xeon w9 (60 cores / 120 threads), 502 GiB RAM (see local `pip freeze` for env details).
- Note: For model inference with GPUs, configure CUDA and specify GPU devices; current repo defaults to CPU-friendly libraries unless GPU-enabled models are pulled.

8. Reproducibility Checklist (what to archive)

- `data/vector_index/*` (FAISS indexes and docs.jsonl) — essential for retrieval experiments.
- `data/analytics/*` used for analysis and plots.
- The `logs/activity_log.json` slices used for evaluation.
- `requirements.txt` pinned via `pip freeze > requirements.txt` and optionally a conda environment YAML.
- Ollama model identifiers and image digests for any local models used.
- Evaluation scripts and sample annotations (CSV/JSONL) with annotation guidelines.

9. Commands and Quick Recipes

- Rebuild and run Streamlit locally (in background):

```
source .venv/bin/activate
streamlit run chatgpt.py
```

- Pin dependencies (recommended before sharing artifacts):

```
pip freeze > requirements.txt
```

- Pull common Ollama models (example):

```
# Requires Ollama installed and running
ollama pull llama3.2:3b
ollama pull qwen3:4b
```

10. Data Management and Ethics

- Anonymize user identifiers when sharing datasets in publications.
- Remove PII from `logs/activity_log.json` before archiving.
- Document consent and IRB approvals where required.

11. Where Code Lives

- RAG retriever: `lib/rag.py`
- Generation wrapper: `lib/chatgeneration.py`
- Instructor dashboard: `lib/home_views.py`
- Analytics scripts: `scripts/compute_counts.py`, `scripts/compute_topic_difficulty.py`, `scripts/compute_errors.py`, `scripts/classify_topics_ollama.py`

Notes and next steps: - If you want, I can also produce a LaTeX-ready Methods section file, a PDF export, or an `experiments/` runner with example configs and test harness. Tell me which you'd prefer next.